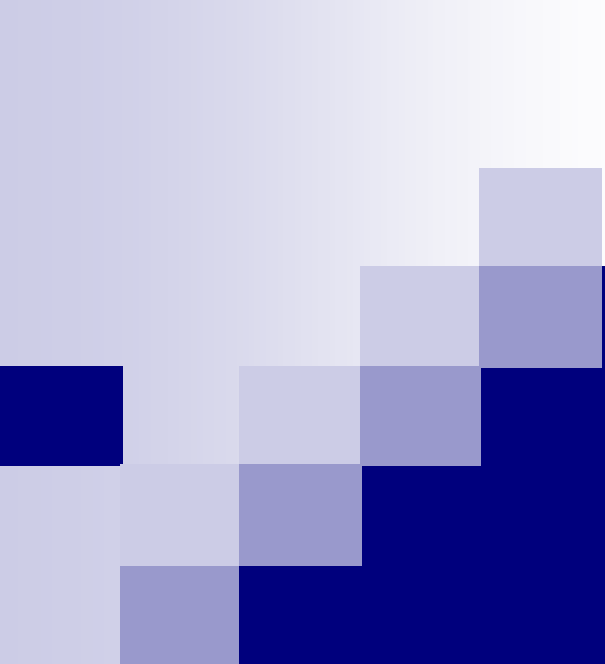




Lecture 1

Introduction

Ying CAO
SIST, ShanghaiTech
Spring, 2026

Outline

- Course logistics
 - Overall objective
 - Pre-requisite/Syllabus
 - Assessment/Grading
- Introduction to deep learning
- Machine learning review
- Artificial neurons

Teaching Team

■ Instructor:

- Prof. Ying CAO
- caoying1@shanghaitech.edu.cn
- Room 1C-403C, SIST Building

■ 3 Teaching assistants

■ We will use Piazza as the main communication platform

- <https://piazza.com/shanghaitech.edu.cn/spring2026/cs280/info>

Course objectives

- Understand deep learning models and the training methods
 - Key concepts, fundamental principles, and underlying mechanisms
 - What is happening inside the models
- Learn to implement deep learning models
 - How to write from scratch, debug and train neural networks
 - How to implement neural networks to solve practical problems
- Learn to apply deep learning models in various fields
 - Computer Vision, Natural Language Processing (NLP), etc.

Pre-requisite

- Calculus, Linear Algebra, Probability and Statistics
 - Undergraduate course level
- Basic knowledge of Machine Learning
 - Formulating cost functions
 - Taking derivatives
 - Performing optimization with gradient descent
- Proficiency in Python
 - All class assignments will be in Python
- Will be evaluated in next quiz (this Thursday)

Syllabus

- Part I: Basic Neural Networks
 - Single-Layer Networks
 - Multiple-Layer Networks
 - Gradient Descent and Backpropagation (BP)
- Part II: Convolutional Neural Networks
- Part III: Recurrent Neural Networks
- Part IV: Transformers
- Part V: Deep Generative Models

Syllabus

- Part I: Basic Neural Networks
- Part II: Convolutional Neural Networks
 - CNN Basics
 - Understanding CNN
 - CNN in Vision
- Part III: Recurrent Neural Networks
- Part IV: Transformers
- Part V: Deep Generative Models

Syllabus

- Part I: Basic Neural Networks
- Part II: Convolutional Neural Networks
- Part III: Recurrent Neural Networks
 - RNN basics and BP through Time
 - LSTM, GRU
 - Attention Mechanism
 - RNN in Vision and NLP
- Part IV: Transformers
- Part V: Deep Generative Models

Syllabus

- Part I: Basic Neural Networks
- Part II: Convolutional Neural Networks
- Part III: Recurrent Neural Networks
- Part IV: Transformers
 - Self-attention
 - Transformer Architecture
 - Transformers in Vision and NLP
- Part V: Deep Generative Models

Syllabus

- Part I: Basic Neural Networks
- Part II: Convolutional Neural Networks
- Part III: Recurrent Neural Networks
- Part IV: Transformers
- Part V: Generative Neural Networks
 - Variational Autoencoders (VAE)
 - Generative Adversarial Networks (GAN)
 - Diffusion Models (DM)

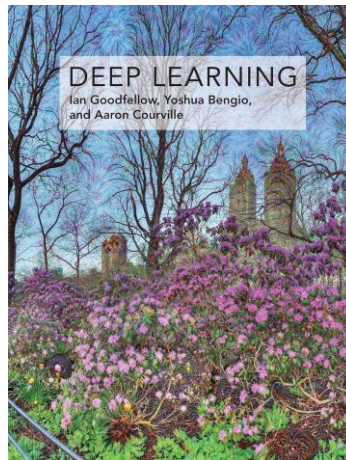
Reference books and materials

■ Deep learning books:

- <http://www.deeplearningbook.org/>
- <https://d2l.ai/>

■ Online deep learning courses:

- Stanford: CS230, CS231n
- CMU: 11-785
- MIT: 6.S191



Assessment and Grading

- 9 Quizzes (in class): 30%
 - 2% for the first quiz, 3.5% for each of the others
- 4 Assignments: 30%
- Final course project: 40%
 - Any topic that you are interested in
 - Project team: 4~5 students
 - Project proposal (5 mins) + Final report (paper format) + Final presentation (10 mins)
 - Grading is based on the amount of work done *for this course* (datasets, approaches, applications, etc.)
 - Grading on each member is based on the contribution
- Late policy
 - A total of 7 free late days to use, but no more than 4 late days can be used on any single assignment.
 - After that, 25% off per day late
 - Does not apply to the final course project and quizzes

Academic integrity

■ Plagiarism

- All assignments must be done individually
 - You may not look at solutions from any other source
 - You may not share solutions with any other students
 - You may discuss together or help another student but you cannot give the exact solution

■ Plagiarism punishment

- When one student copies from another, both students will be held responsible
- Zero point on the assignment or quiz in question
- Repeated violation will result in an F grade for this course as well as further discipline at the school/university level

Outline

- Course logistics
- Introduction to deep learning
 - What & Why deep learning?
- Machine learning review
- Artificial neurons

Acknowledgement: Bhiksha Raj's course notes

Introduction

- Our goal: Build intelligent algorithms to make sense of data to solve various real-world tasks

- Example: Recognize objects in images



Red panda

- Example: Predict what would happen next



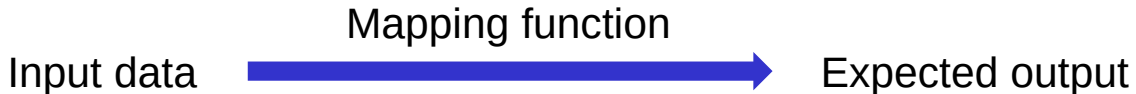
Handshake

Introduction

- A broad range of real-world applications
 - Image classification
 - Input: images $\rightarrow$ Output: image category (cat, dog, car, house, etc.)
 - Language translation
 - Input: text in language A $\rightarrow$ Output: text in language B
 - Speech recognition
 - Input: sound wave $\rightarrow$ Output: text
- Challenges: difficult to manually design algorithms
 - What features and rules to use?

A data-driven approach

- Each task as a mapping function (or a model)



- Input data: images
- Expected output: object names

- Learn the mapping function from data



Red panda



Mapping function

A data-driven approach

- Build a mapping function (model): $y = f(x; \theta)$
 - x : input data
 - y : expected output
 - θ : parameters to be estimated
- Learn the model from data
 - Given a dataset $\{(x_n, y_n)\}_{n=1}^N$
 - Find the “best” parameter $\hat{\theta}$, such that $y_n = f(x_n; \hat{\theta}) \quad \forall n$
 - It can be generalized to unseen input data

What is deep learning?

- Using a deep neural network as the mapping function
- Model: Deep neural networks
 - A family of parametric models
 - Consist of many “simple” computational units
 - Have multiple layers of connected units to process data hierarchically

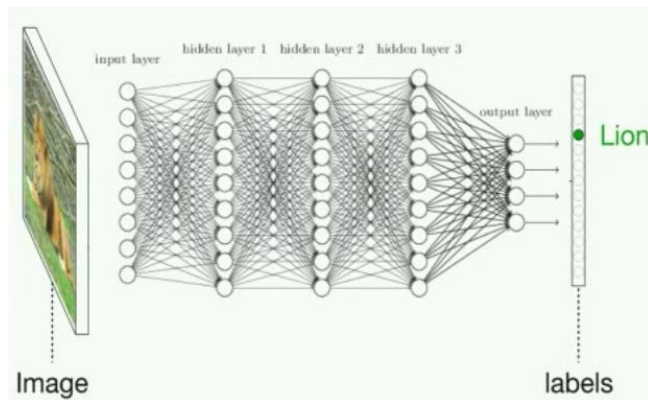


Image from Jeff Clune's Deep Learning Overview

What is deep learning?

- Using a deep neural network as the mapping function
- Learning: Parameter estimation from data
 - Parameters: **connection weights between units**
 - Formulated as an **optimization** problem
 - Efficient algorithms for handling **large models & datasets**

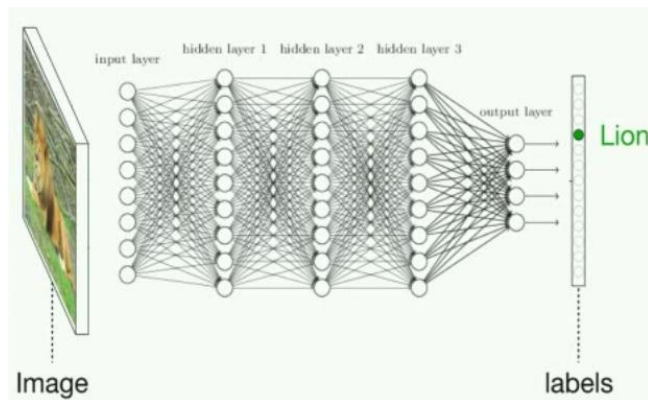
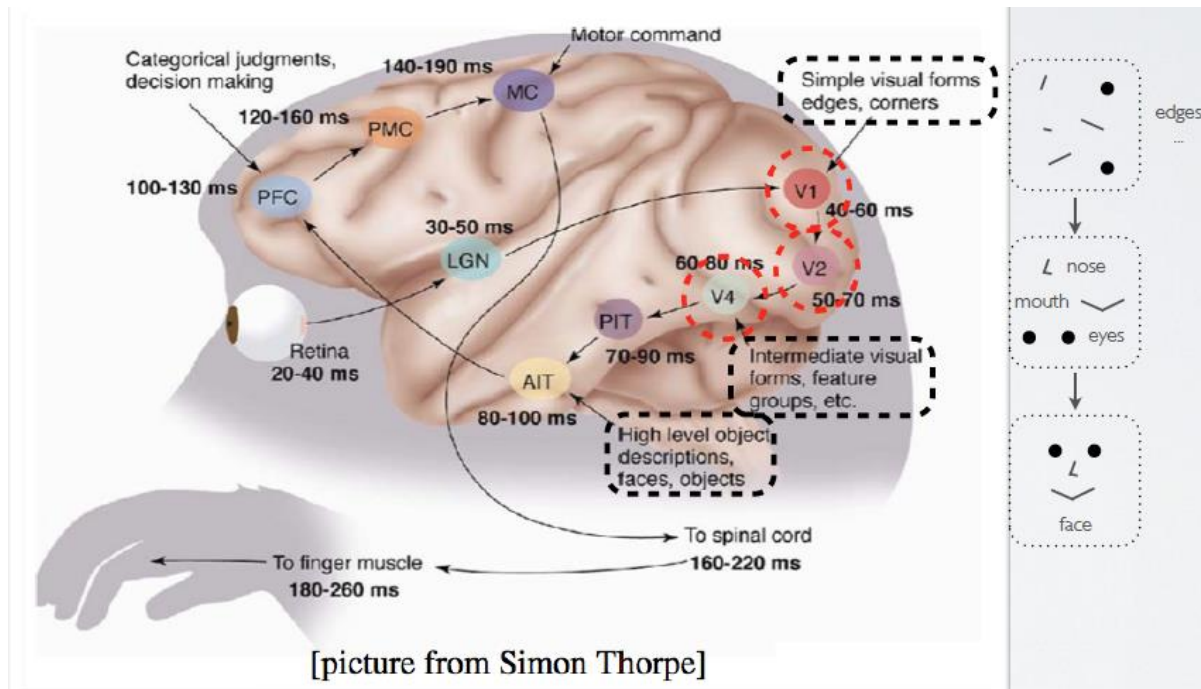


Image from Jeff Clune's Deep Learning Overview

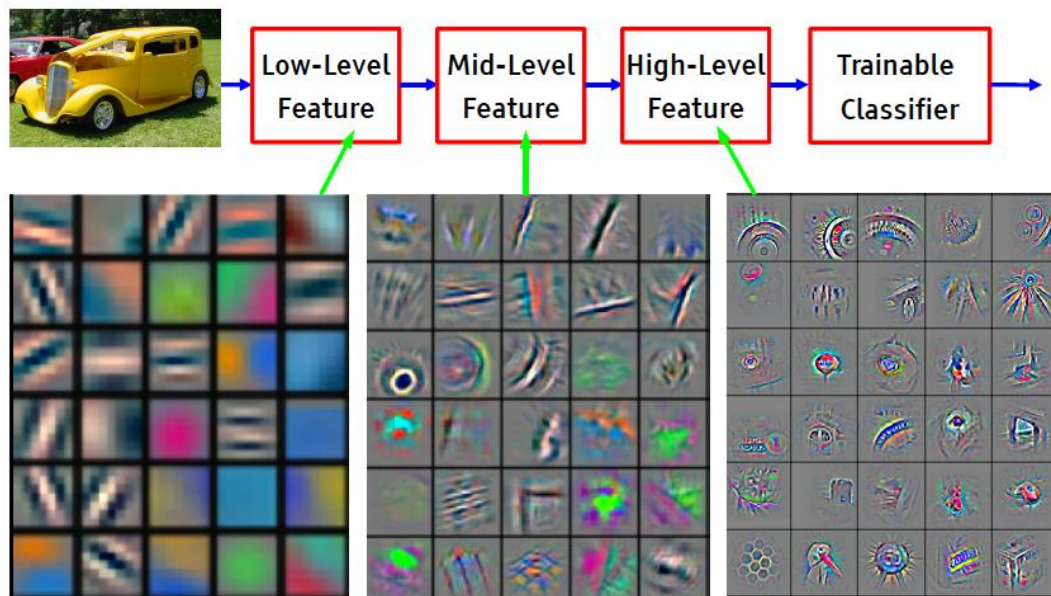
What is deep learning?

■ Inspiration from visual cortex



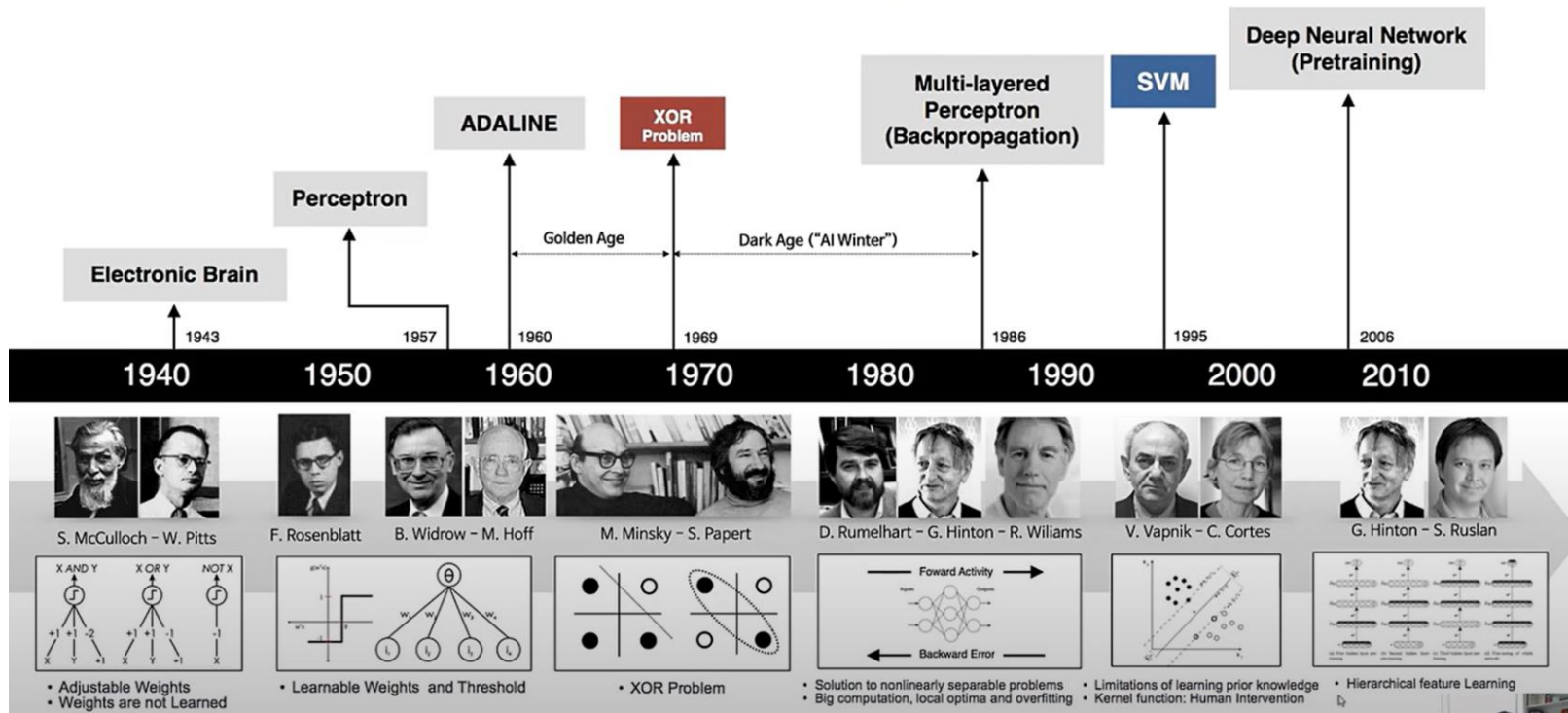
Why deep networks?

- A deep architecture can represent any function *compactly*
- Learn an effective, hierarchical representation of data *automatically*



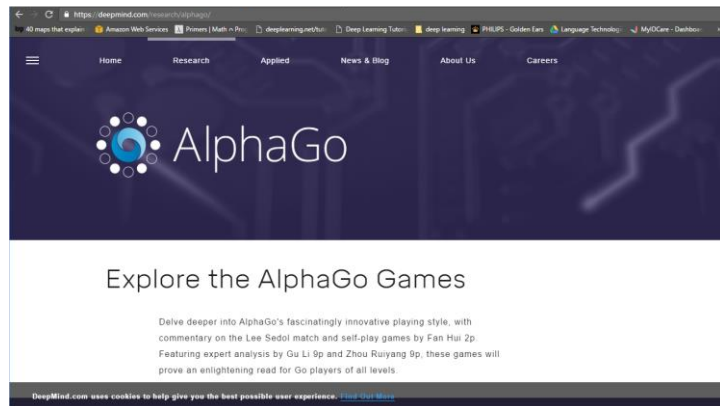
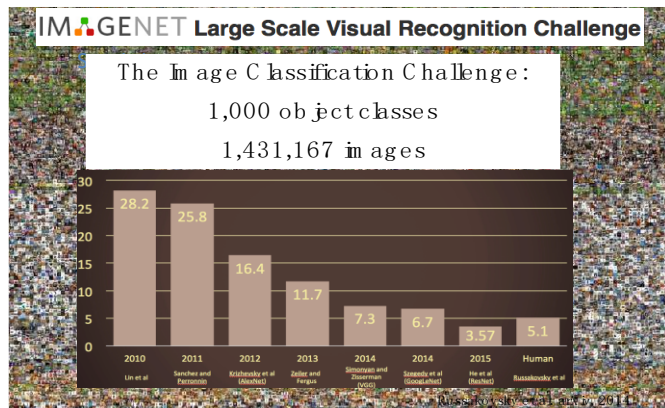
Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Deep learning history



Recent success with DL

- AlexNet on ImageNet (2012) and AlphaGo (2016)



- Since 2016, DL becomes very popular and starts to revolutionize various fields, e.g., CV, NLP, speech recognition and CG

Recent success with DL

■ Latest breakthrough in generative AI since 2022

□ Scaling deep models (Transformers, diffusion models) on Internet-scale data

Can you explain what the function of DNA is in the style of a rapper?



Sure, here's a little something I came up with:

DNA, it's the code that we're made of
It tells our cells what to do, like a boss
It's a double helix, twisted and coiled
And inside each one, our genetic info is toiled

It holds the instructions for every trait we possess
From the color of our eyes to the shape of our chest
It determines our height, our hair, and our skin
It's the blueprint for who we are, from within

So when you're chillin' on the block, or rockin' on the mic
Remember that your DNA is the reason you're so fly
It's the boss of your body, the king of your cells
Without it, you wouldn't be you, you'd just be empty shells

So give it some respect, and take good care of it
Your DNA is the most important part of your kit
It's the code that makes you, the real you
So take care of it, and your future will be bright, true!

Text generation
(e.g., ChatGPT)

“a photograph of an astronaut riding a horse”



Image generation
(e.g., Stable Diffusion)

“A white and orange tabby cat is seen happily darting through a dense garden, as if chasing something...”



Video generation
(e.g., Sora)

Recent success with DL



ACM Turing Award 2018 (Nobel Prize of Computing)
Yann LeCun, Geoffrey Hinton, and Yoshua Bengio

Summary: Why deep learning?

- Powerhouse behind latest breakthrough in different fields
 - Other key factors: larger datasets and better computing hardware
- State of the art results in many problems
 - Often exceed traditional methods by large margins
 - Surpass human performance for certain “complex” tasks
- But also somewhat controversial
 - Lack of theoretical understanding
 - Sometimes difficult to make it work in practice

Outline

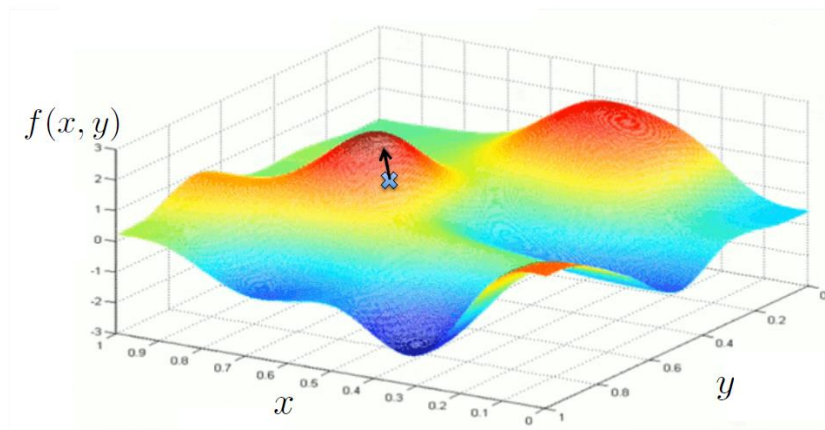
- Course logistics
- Introduction to deep learning
- Machine learning review
 - Math review
 - Supervised learning
- Artificial neurons

Acknowledgement: Hugo Larochelle's, Mehryar Mohri's & Yingyu Liang's course notes

Math review: Calculus

■ Gradient

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \left[\frac{\partial}{\partial x_1} f(\mathbf{x}), \dots, \frac{\partial}{\partial x_d} f(\mathbf{x}) \right]^\top = \begin{bmatrix} \frac{\partial}{\partial x_1} f(\mathbf{x}) \\ \vdots \\ \frac{\partial}{\partial x_d} f(\mathbf{x}) \end{bmatrix}$$



Math review: Probability

■ Probability chain rule

$$p(s, o) = p(s|o)p(o) = p(o|s)p(s)$$

□ In general:

$$p(\mathbf{x}) = \prod_i p(x_i | x_1, \dots, x_{i-1})$$

■ Bayes rule

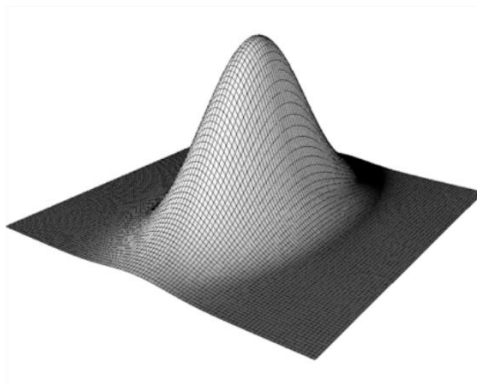
$$p(o|s) = \frac{p(s|o)p(o)}{p(s)}$$

Math review: Probability

■ Gaussian distribution

□ Random variable: $\mathbf{x} \in \mathbb{R}^d$

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma)}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$



Math review: Statistics

■ Monte Carlo estimation

- A method to approximate an expectation

$$E_{\mathbf{x} \sim p(\mathbf{x})}[f(\mathbf{x})] \approx \frac{1}{K} \sum_k^K f(\mathbf{x}_k)$$

- $\mathbf{x}_k$ is sampled from $p(\mathbf{x})$

■ Maximum likelihood

$$\hat{\theta} = \arg \max_{\theta} p_{\theta}(\mathbf{x}_1, \dots, \mathbf{x}_N)$$

- Independent and identically distributed (i.i.d.): $p_{\theta}(\mathbf{x}_1, \dots, \mathbf{x}_N) = \prod_i^N p_{\theta}(\mathbf{x}_i)$

Standard learning problems

- **Supervised learning**

- ☐ Use *labeled* data for learning

- **Unsupervised learning**

- ☐ Use *unlabeled* data for learning

- **Semi-supervised learning**

- ☐ Use both *unlabeled* and *labeled* data for learning

- **Reinforcement learning**

- ☐ Use environment feedback (*reward*) for learning

- ...

Supervised learning

■ Problem setup

- Given training data $\{(x_i, y_i)\}_{i=1}^n$
- Find $f: x \rightarrow y$ using training data
- s.t. f has high performance on test data

Supervised learning

■ Problem setup

- Given training data $\{(x_i, y_i)\}_{i=1}^n$
- Find $f: x \rightarrow y$ using training data
- s.t. f has high performance on test data

What kind of functions?

Supervised learning

■ Problem setup

- Given training data $\{(x_i, y_i)\}_{i=1}^n$
- Find $f: x \rightarrow y \in \mathcal{H}$ using training data
- s.t. f has high performance on test data



Hypothesis class

Supervised learning

■ Problem setup

- Given training data $\{(x_i, y_i)\}_{i=1}^n$
- Find $f: x \rightarrow y \in \mathcal{H}$ using training data
- s.t. f has high performance on test data

Relationship between
training and test data?

Supervised learning

■ Problem setup

- Given training data $\{(x_i, y_i)\}_{i=1}^n$ (i.i.d. from distribution D)
- Find $f: x \rightarrow y \in \mathcal{H}$ using training data
- s.t. f has high performance on test data (i.i.d. from distribution D)



They come from the same distribution

Supervised learning

■ Problem setup

- Given training data $\{(x_i, y_i)\}_{i=1}^n$ (i.i.d. from distribution D)
- Find $f: x \rightarrow y \in \mathcal{H}$ using training data
- s.t. f has high performance on test data (i.i.d. from distribution D)

How to measure performance?

Supervised learning

■ Problem setup

- Given training data $\{(x_i, y_i)\}_{i=1}^n$ (i.i.d. from distribution D)
- Find $f: x \rightarrow y \in \mathcal{H}$ using training data
- s.t. the expected loss is small

$$L(f) = \mathbb{E}_{(x, y) \sim D} [l(f, x, y)]$$

Supervised learning

■ Problem setup

- Given training data $\{(x_i, y_i)\}_{i=1}^n$ (i.i.d. from distribution D)
- Find $f: x \rightarrow y \in \mathcal{H}$ using training data
- s.t. the expected loss is small

$$L(f) = \mathbb{E}_{(x, y) \sim D} [l(f, x, y)]$$

Examples of loss function l :

- 0-1 loss: $l(f(x), y) = \mathbb{I}[f(x) \neq y]$
- l_2 loss: $l(f(x), y) = [f(x) - y]^2$

Supervised learning

■ Problem setup

- Given training data $\{(x_i, y_i)\}_{i=1}^n$ (i.i.d. from distribution D)
- Find $f: x \rightarrow y \in \mathcal{H}$ using training data
- s.t. the expected loss is small

$$L(f) = \mathbb{E}_{(x, y) \sim D} [l(f, x, y)]$$

How to use the loss for learning?

Supervised learning

■ Problem setup

- Given training data $\{(x_i, y_i)\}_{i=1}^n$ (i.i.d. from distribution D)
- Find $f: x \rightarrow y \in \mathcal{H}$ that minimizes $L(f) = \frac{1}{n} \sum_{i=1}^n l(f, x_i, y_i)$
- s.t. the expected loss is small

$$L(f) = \mathbb{E}_{(x, y) \sim D} [l(f, x, y)]$$

Empirical loss

Learning as iterative optimization

■ Gradient descent (GD) to optimize function parameters w

- ▶ Choose initial w_0 , repeat

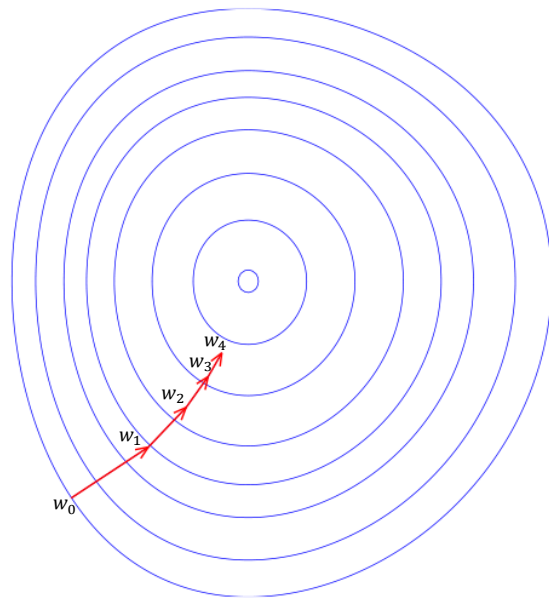
$$w_{t+1} = w_t - \eta \nabla_{w_t} L(w_t)$$

until stop

- ▶ η is the learning rate, and

$$\nabla_{w_t} L(w_t) = \frac{1}{n} \sum_{i=1}^n \nabla l_{w_t}(w_t, x_i, y_i)$$

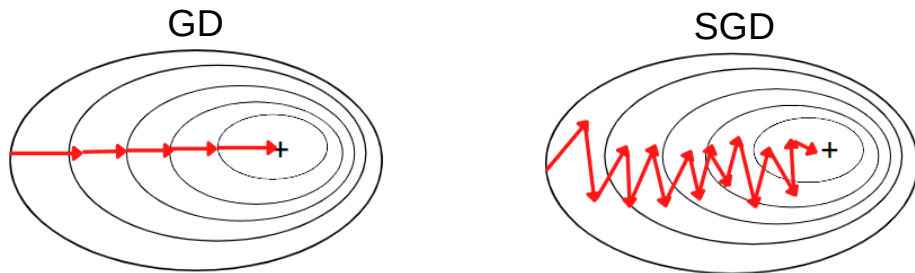
- ▶ When to stop? $\|w_{t+1} - w_t\| \leq \epsilon$ or $\|\nabla_{w_t} L(w_t)\| \leq \epsilon$
- ▶ It is **slow** since every parameter update uses all training examples



Learning as iterative optimization

■ Stochastic gradient descent (SGD)

- ▶ Suppose data points arrive one by one
- ▶ $L(w_t) = \frac{1}{n} \sum_{i=1}^n l(w_t, x_i, y_i)$, but we only know $l(w_t, x_i, y_i)$ at time t
- ▶ Idea: do what you can based on local information
 - Initialize w_0
 - $w_{t+1} = w_t - \eta \nabla l_{w_t}(w_t, x_i, y_i)$
- ▶ It is **fast but instable**



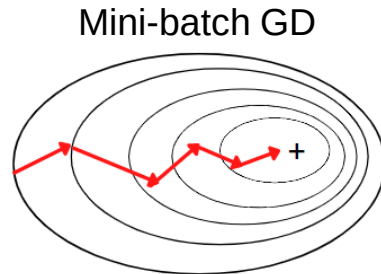
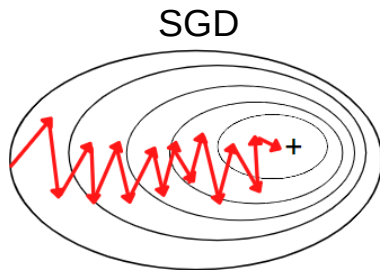
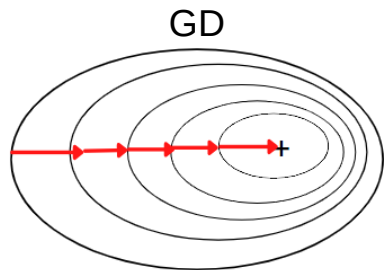
Learning as iterative optimization

■ Mini-batch gradient descent

- ▶ Perform parameter update with a mini-batch of m training examples:

$$L(w_t) = \frac{1}{m} \sum_{i=1}^m l(w_t, x_i, y_i)$$

- ▶ m is batch size ($\ll n$)
- ▶ Combine the advantages of GD and SGD: faster than GD; more stable than SGD



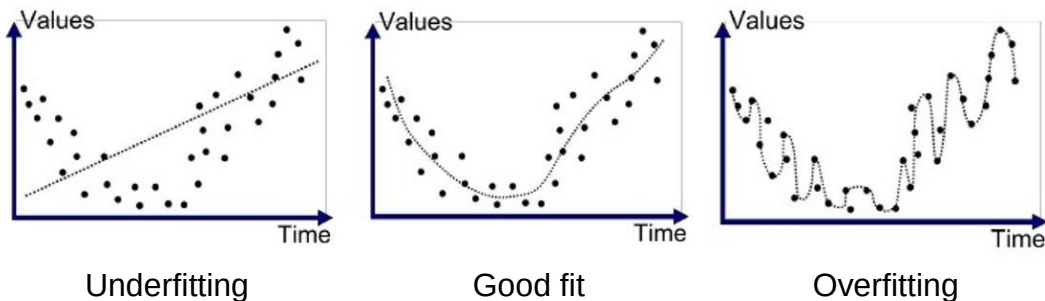
Underfitting vs. Overfitting

■ Underfitting

- The model is too simple so that it fails to fit training data well
- High training error, high test error

■ Overfitting

- The model is too complex so that it fits noise in training data well
- Low training error, high test error



Outline

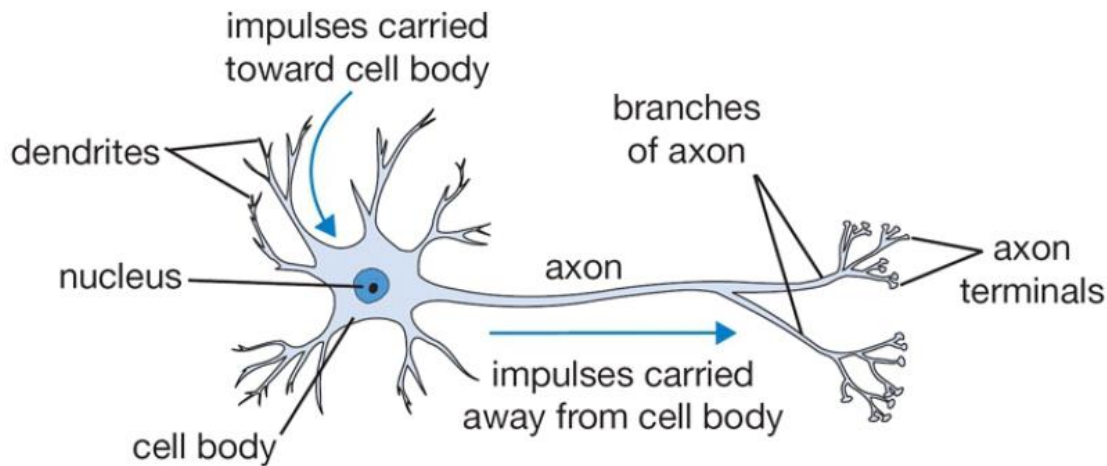
- Course logistics
- Introduction to deep learning
- Machine learning review
- Artificial neurons
 - Math model
 - Perceptron model

Acknowledgement: Hugo Larochelle's, Mehryar Mohri's & Yingyu Liang's course notes

Artificial neuron

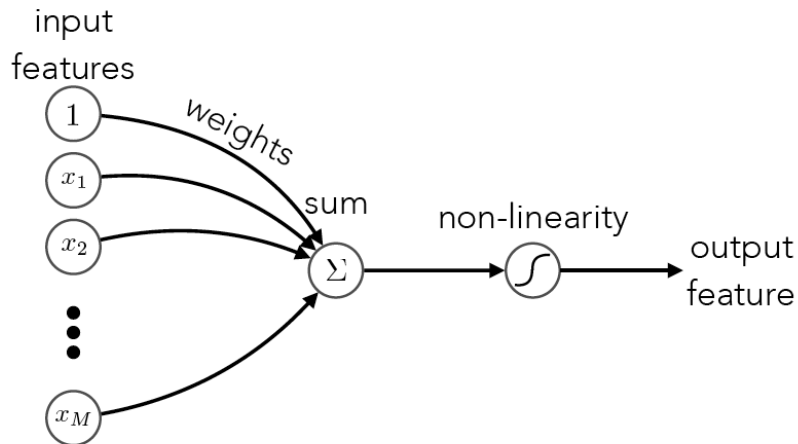
■ Biological inspiration

- Our brain has $\sim 10^{11}$ neurons, each of which communicates (is connected) to many other neurons



The basic computation unit of the brain: Neuron

Mathematical model of a neuron



artificial neuron: *weighted sum and non-linearity*

$$s = \overset{\text{bias}}{b} + \overset{\text{weights}}{w_1}x_1 + \overset{\text{input features}}{w_2}x_2 + \cdots + \overset{\text{input features}}{w_M}x_M = \mathbf{w}^T \mathbf{x}$$

$h = g(s)$

output feature non-linearity sum

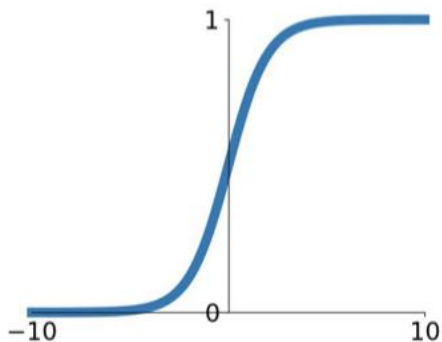
The diagram shows the equation $s = b + w_1x_1 + w_2x_2 + \cdots + w_Mx_M = \mathbf{w}^T \mathbf{x}$ with dotted arrows pointing from the labels "bias", "weights", and "input features" to their respective terms. Below the equation, the output is given as $h = g(s)$, with dotted arrows pointing from "output feature", "non-linearity", and "sum" to their respective parts.

Activation functions

- Most commonly used activation functions (non-linearities):

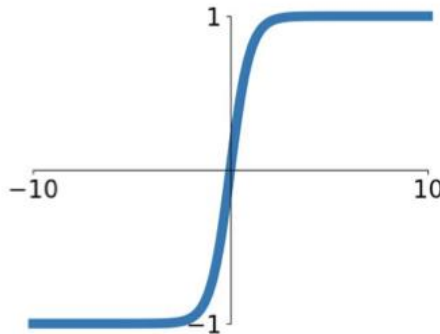
Sigmoid:

$$\sigma(x) = 1/(1 + e^{-x})$$



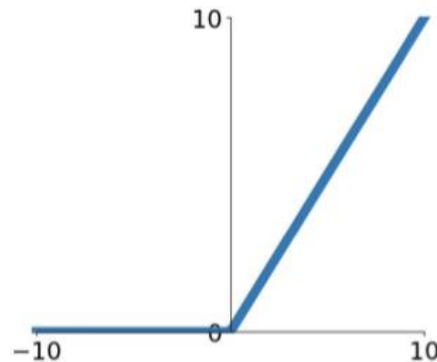
Tanh:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



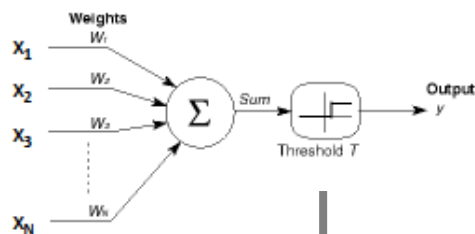
Rectified Linear Unit:

$$\text{ReLU}(x) = \max(0, x)$$



What a single neuron does?

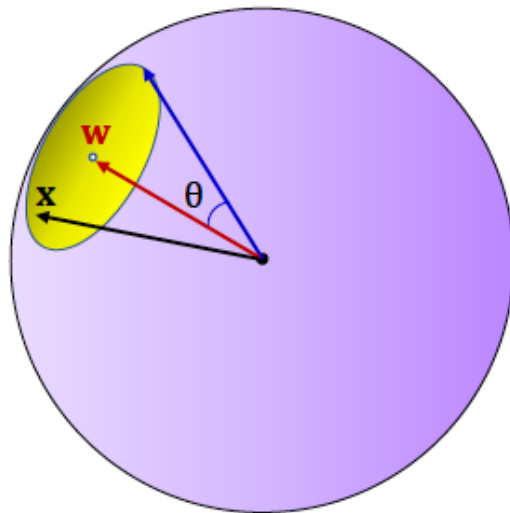
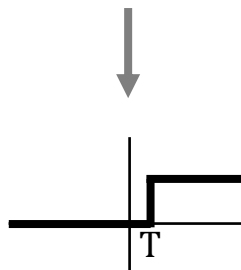
- A neuron fires if the angle between its input vector and weight vector is smaller than a certain value



$$\mathbf{w}^T \mathbf{x} > T$$

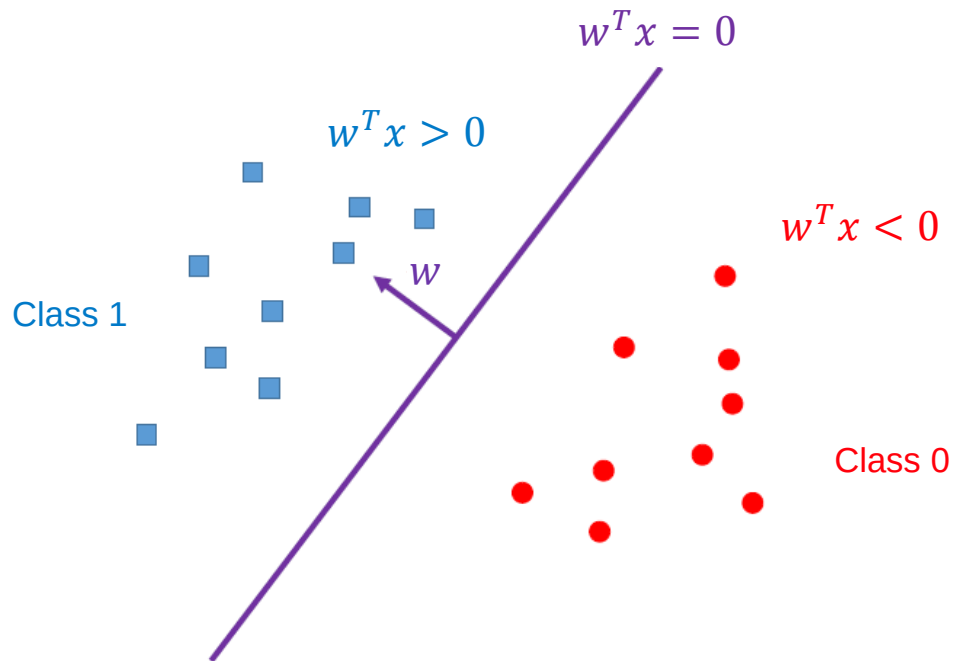
$$\Rightarrow \cos \theta > \frac{T}{\|\mathbf{w}\| \|\mathbf{x}\|}$$

$$\Rightarrow \theta < \cos^{-1} \left(\frac{T}{\|\mathbf{w}\| \|\mathbf{x}\|} \right)$$



Single neuron as a linear classifier

- Perceptron model for binary classification



Summary

- Course logistics
 - Introduction to deep learning
 - Review of basic math & ML
 - Artificial neurons
-
- Next time
 - Basic neural networks
 - First quiz on prerequisite